

UE28 Big data

Data engineering | MLOps

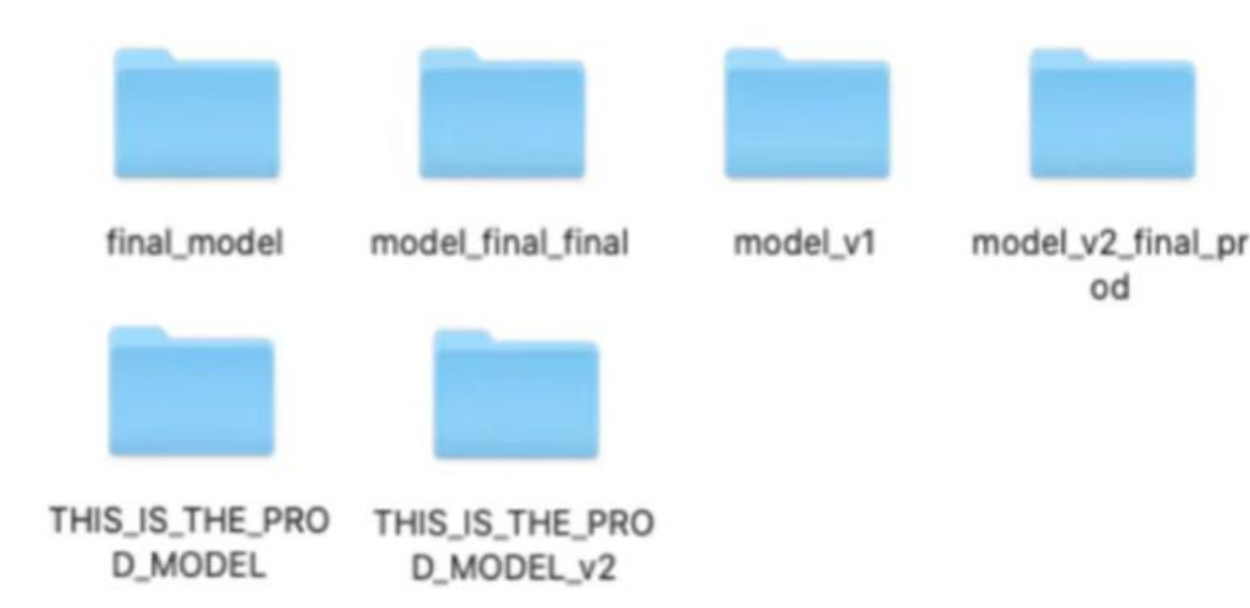
INFORMATIQUE Intelligence Artificielle

BLOC 2 | Q2 | Linda Wang

Pourquoi structurer son projet data/IA ?

- ...
- Un humain doit s'y retrouver sans peine.
- Une IA doit s'y retrouver sans peine.
- Programmer l'ordre de lancement des scripts/pipelines
- Versionner le code
- Versionner les données
- Versionner les modèles
- Observabilité et reproductivité des entraînements de modèles

Pourquoi structurer son projet data/IA ?



Quelles bonnes pratiques recommandez-vous ?

• ...

Scaffolding

Comment créer un projet template ?

💡 Utilisons un outil de scaffolding

- Copier
- Cookiecutter
- Github Templates

+ Maintainable



+ Facile à utiliser



Scaffolding

	Copier	Cookiecutter	GitHub Templates
Type	Template engine moderne	Template engine classique	Template repo
Complexité	Moyenne	Moyenne	Très faible
Variables interactives	✓	✓	✗
Mise à jour du projet	✓	✗	✗
Versionning des templates	✓	⚠ manuel	✗
Écosystème principal	Python / généraliste	Python / généraliste	Tous
Facilité d'usage	★ ★ ★ ★	★ ★ ★ ★	★ ★ ★ ★ ★
Maintenance long terme	★ ★ ★ ★ ★	★ ★ ★	★ ★
Adoption actuelle	En forte croissance	Stable	Très élevée
State-of-the-art (2026)	Oui (avancé)	Partiellement	Oui (simple)

Claude code as Scaffolding tool

I feel like asking claude code to generate from an empty folder uses a lot of tokens and it's a log of useless files and folders. Is that a common feeling among the developer community ? What can i do to improve that ? I feel like doing the initial setup myself, do claude code `"/init"` and asking claude code to add feature is better

Yes, that's a very common feeling and you're right on all counts.

The community consensus has largely converged on exactly what you're describing. Letting Claude Code `scaffold` from zero tends to produce generic boilerplate it then has to explain, verify, and sometimes fix — burning tokens on things like `README.md` placeholders and empty `tsconfig` files you could create in 30 seconds yourself.

The pattern most experienced Claude Code users settle on:

1. **You do the skeleton manually** — `pnpm create next-app`, `pnpm init` for packages, copy a base `tsconfig`. This takes 5 minutes and you *understand* every file.
2. **Run `/init`** — this lets Claude Code read your actual project, build its own mental map, and write a `CLAUDE.md` that captures your conventions. It's far more useful when there's real structure to analyze rather than an empty folder.
3. **Ask for features, not files** — "Add Supabase auth middleware that protects `/app/*` routes" is a much better prompt than "create the auth system". Claude touches only what's needed, you see exactly what changed, and you learn because the diff is readable.

Claude code as Scaffolding tool

The broader principle the community calls it:

"Claude Code is a senior dev you pair with, not a **Yeoman generator**."

You wouldn't ask a senior dev to run `create-next-app` for you. You'd set up the repo yourself, give them context via `CLAUDE.md`, and then ask them to solve specific problems.

What `CLAUDE.md` buys you specifically:

Once you've done `/init` on a real project, Claude Code stops re-inferring your stack every prompt. It knows you're on pnpm, on Turborepo, that `packages/db` is your Supabase layer, that you stream responses. That context reduction alone cuts token usage significantly on every subsequent feature request.

So your instinct is correct — revised workflow:

You manually:	<code>create-next-app</code> + pnpm workspace setup (~5 min)
Claude Code <code>/init</code> :	reads structure, writes <code>CLAUDE.md</code>
You + Claude Code:	add one feature at a time, iteratively

Much cheaper, much more educational, and you stay in the driver's seat.

Yeoman est un outil de scaffolding (comme Copier ou Cookiecutter) populaire pour le développement web.

Que mettre dans mon template ?

Plusieurs paradigmes existent.

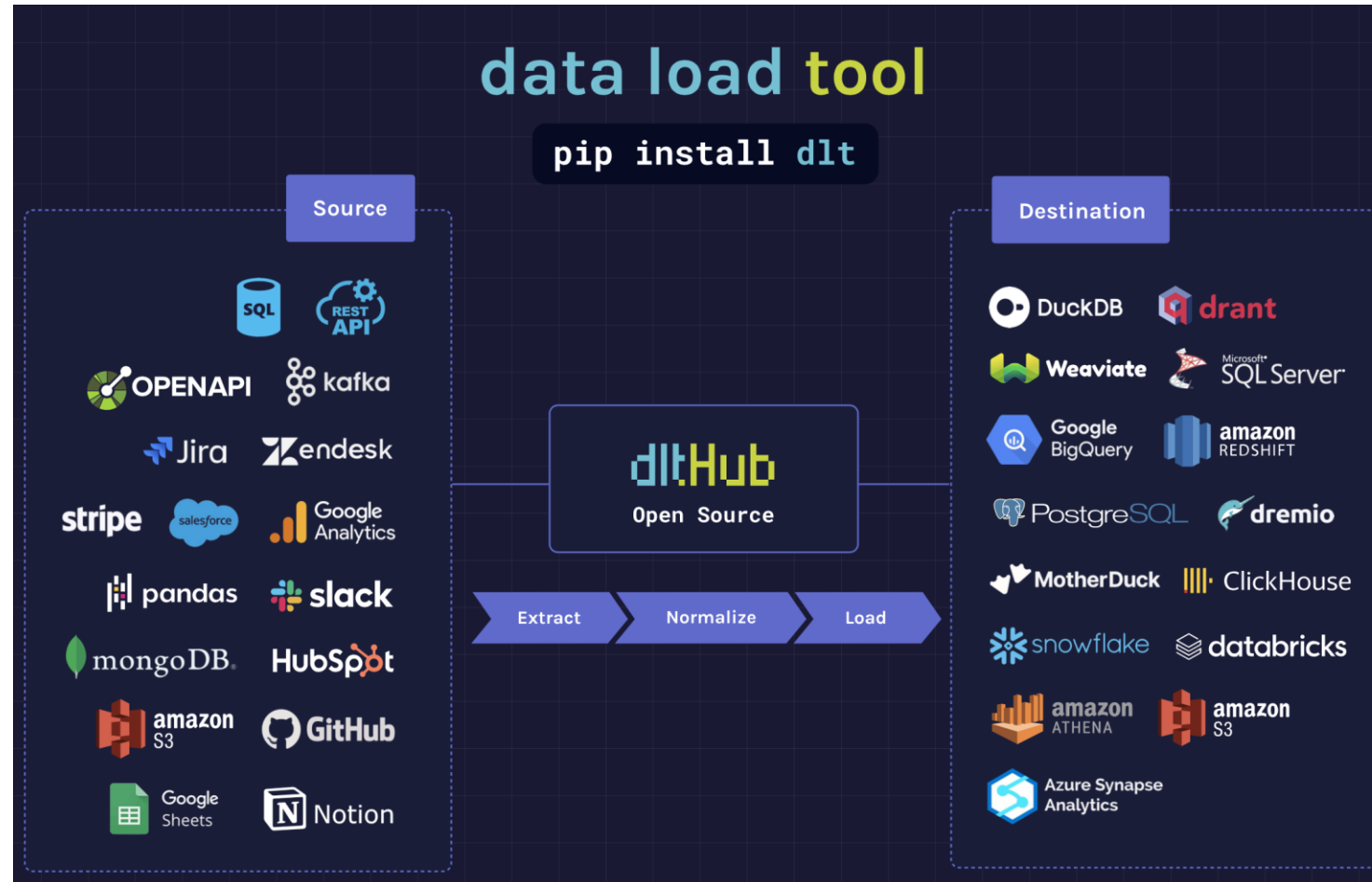
Je vous présente **5 librairies**. Voyez-les comme des sources d'inspiration.

Chaque librairie a son paradigme qui comporte ses avantages et ses inconvénients.

👉 A vous de voir ce qui **convient** le mieux pour votre projet et comment vous **préférez** travailler.

Librairie utile #1

ditHub



Description

Une bibliothèque Python légère pour construire des **pipelines ELT** (Extract-Load-Transform). dlt permet de charger des données depuis n'importe quelle API avec une spec OpenAPI sans écrire de code, et supporte des formats comme Parquet, Delta Tables ou Iceberg pour créer facilement ses propres data lakes.

Applications

- ETL
- Data warehousing
- Ingérer données venant d'API
- ⚠️ Peu compatible avec Spark
- ⚠️ Pas pour ML, ni pour GenAI/LLM

Mais peut être intéressant pour RAG si vos données viennent d'API.

dlt - Resource

```
import pandas as pd
```

```
# Define a resource to load data from a CSV
```

```
@dlt.resource(table_name="df_data")
```

```
def my_df() -> TDataItems:
```

```
    sample_df = pd.read_csv("https://people.sc.fsu.edu/~jburkardt/data/csv/hw_200.csv")
```

```
    yield sample_df
```

```
# Run the pipeline with the defined resource
```

```
load_info = pipeline.run(my_df)
```

```
print(load_info)
```

```
# Query the loaded data from 'df_data'
```

```
pipeline.dataset().df_data.df()
```

dlt - Source

Une **source** dans DLT est l'unité d'organisation principale du pipeline.

Concrètement :

- **Regroupement logique** : une source représente *une seule origine de données*. Elle regroupe plusieurs **ressources** (= les tables individuelles : clients, commandes, paiements...).
- **@dlt.source** : décorateur Python qui signale à DLT que cette fonction est une source. Elle doit retourner une ou plusieurs ressources (@dlt.resource).
- **Schéma optionnel** : noms de colonnes, types, clés primaires, hints pour optimiser les performances (ex: colonnes de partitionnement).
- **Module séparé** : bonne pratique — chaque source vit dans son propre fichier `.py`, ce qui garde le code organisé et réutilisable.
- **Auth + pagination** : c'est dans ce module qu'on met la logique *technique* de connexion à l'API : gestion des tokens, des clés API, et la pagination (récupérer les pages 1, 2, 3... automatiquement).

dlt - Transformer

Librairie utile #2



Démo avec template cookie-cutter

<https://github.com/kedro-org/kedro-starters/tree/main/spaceflights-pyspark>

Kedro

Description

Un framework open-source (maintenu par McKinsey / QuantumBlack) pour structurer des **projets data science et ML** de façon reproductible et modulaire, sous forme de pipelines en DAG.

Applications

- ✓ Big data : haute compatibilité avec Spark 🧐
- ✓ ML traditionnel
- ✗ GenAI/LLM
- ✗ Streaming



Kedro

Avantages

- Data Catalog : abstraction centralisée des sources de données (❤️)
- Config loader (❤️)
- Pipelines déclaratifs en Python (nœuds + pipeline)
- Hooks pour l'extensibilité Plugin
- kedro-mlflow pour le tracking d'expériences
- Kedro-Viz pour la visualisation des pipelines (❤️)

Inconvénients

- Gros boilerplate : La structure imposée (Data Catalog, Hooks, namespaces, conventions de nommage) est puissante mais imposante
- 💡 Inspirez-vous de la philosophie Kedro mais sans forcément l'utiliser

Librairie utile #3



Démo

<https://doc.dvc.org/start>

Description

DVC est un outil open-source pour les projets ML qui fonctionne avec Git pour le versioning des données, du code, des métadonnées, des pipelines et des modèles.

Il est idéal pour le **tracking d'expériences**, la **reproductibilité** et les workflows collaboratifs.

Applications

- ✗ Spark
- ✓ ML traditionnel
- ✗ GenAI/LLM

Librairie utile #4

mlflowTM

Démo

<https://mlflow.org/docs/latest/ml/getting-started/quickstart/>

MLflow

Description

MLflow est une plateforme **open-source** centrée sur le **cycle de vie du Machine Learning** : suivi d'expériences, versioning de modèles, reproductibilité et déploiement.

Applications

- ✓ Big data : haute compatibilité avec Spark 🧐
- ✓ ML traditionnel
- ✓ GenAI/LLM : MLflow vient d'intégrer des fonctionnalités de gestion d'agents et de LLM mais ce n'est pas encore mature.

💡 A combiner avec hyperopt

Fonctionnalités clés ❤️

- MLflow Tracking : log des paramètres, métriques, artefacts à chaque run
- Model Registry : versioning et gestion du cycle de vie des modèles (staging, production)
- MLflow Projects : packaging reproductible du code ML
- MLflow Models : déploiement multi-format (REST API, Spark UDF, etc.)

Enregistrer un model :

```
mlflow.sklearn.log_model(model, "model")
```

```
mlflow.spark.log_model(model, "model")
```

Utiliser un model non-spark:

```
predict_udf = mlflow.pyfunc.spark_udf(spark, model_uri="models:/MonModele/Production")  
df_spark.withColumn("prediction", predict_udf("feature1", "feature2", "feature3"))
```

Définitions

Model Signature

La **signature** définit le format attendu des entrées, sorties et paramètres d'un modèle. C'est un **contrat** qui spécifie exactement quelles données le modèle attend en entrée et ce qu'il retournera en sortie.

Flavor

Un **flavor** est une **enveloppe dédiée à une bibliothèque de machine learning spécifique**. Par exemple, le package spark-ml, bien qu'il produise des types de modèles variés comme Pipeline, LogisticRegressionModel ou RandomForestModel, relève tous du flavor Spark. MLflow

Cette abstraction garantit que, quelle que soit la structure interne du modèle, sa variante flavor peut être sauvegardée, loggée et récupérée de manière uniforme via les utilitaires MLflow. MLflow

En pratique, MLflow propose des flavors standards (sklearn, pytorch, tensorflow, spark, etc.) ainsi qu'un flavor universel : **pyfunc**, qui permet de charger n'importe quel modèle comme une fonction Python générique, indépendamment du framework d'origine.

Logging

```
# Start an MLflow run
with mlflow.start_run():
    # Log the hyperparameters
    mlflow.log_params(params)

    # Train the model
    lr = LogisticRegression(**params)
    lr.fit(X_train, y_train)

    # Log the model
    model_info = mlflow.sklearn.log_model(sk_model=lr, name="iris_model")

    # Predict on the test set, compute and log the loss metric
    y_pred = lr.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    mlflow.log_metric("accuracy", accuracy)

    # Optional: Set a tag that we can use to remind ourselves what this run was for
    mlflow.set_tag("Training Info", "Basic LR model for iris data")
```

Auto-logging

Logging automatique de : metrics, parameters, model signature, artifacts, dataset

Librairies supportées :

- [Keras/TensorFlow](#)
- [LightGBM](#)
- [Paddle](#)
- [PySpark](#)
- [PyTorch](#)
- [Scikit-learn](#)
- [Spark](#)
- [Statsmodels](#)
- [XGBoost](#)

Auto logging is a powerful feature that allows you to log metrics, parameters, and models without the need for explicit log statements. All you need to do is to call `mlflow.autolog()` before your training code.

```
python

import mlflow

mlflow.autolog()

with mlflow.start_run():
    # your training code goes here
    ...
```

Model management

Directory written by `mlflow.sklearn.save_model(model, "model", input_example=...)`

model/

├─ MLmodel

├─ model.pkl

├─ conda.yaml

├─ python_env.yaml

├─ requirements.txt

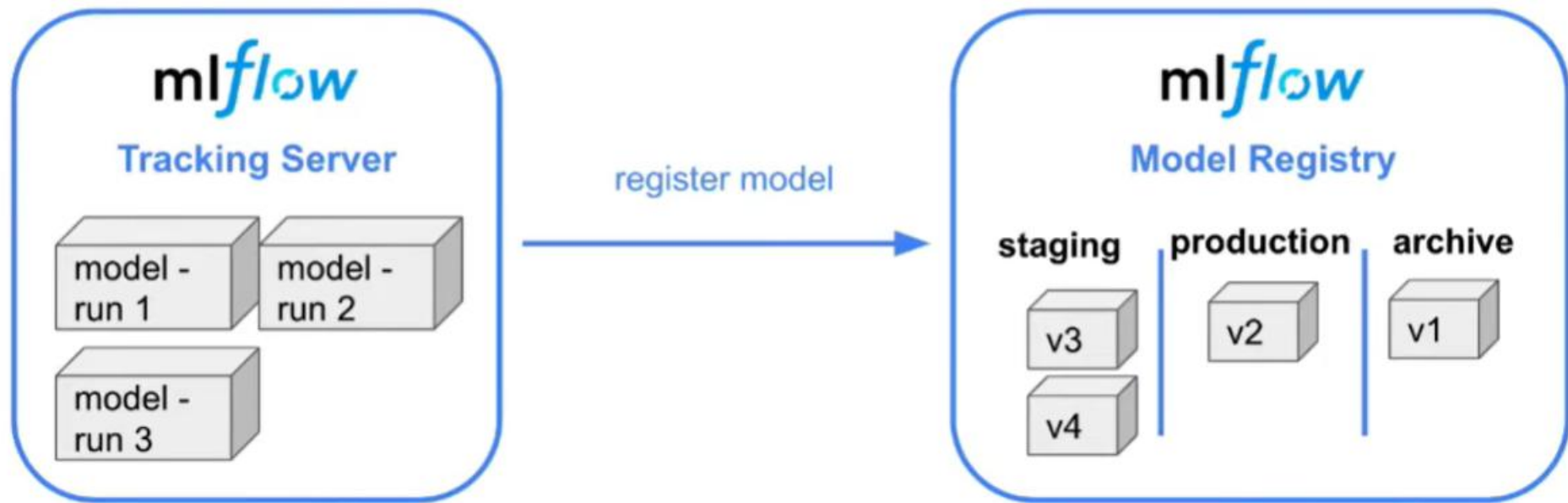
├─ input_example.json (optional, only logged when input example is provided and valid during model logging)

├─ serving_input_example.json (optional, only logged when input example is provided and valid during model logging)

└─ environment_variables.txt (optional, only logged when environment variables are used during model inference)

Model registry

Déployer les modèles vers la prod et partager son utilisation entre développeurs



Model registry

Directory written by `mlflow.sklearn.save_model(model, "model", input_example=...)`

model/

├─ MLmodel

├─ model.pkl

├─ conda.yaml

├─ python_env.yaml

├─ requirements.txt

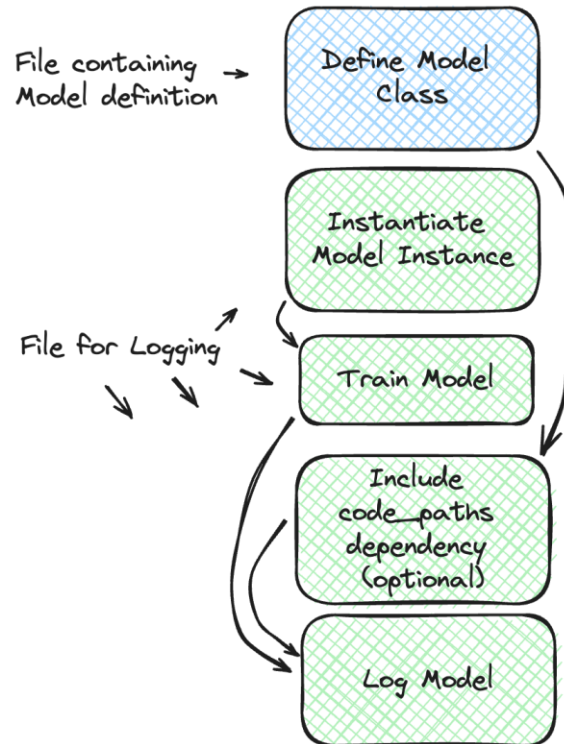
├─ input_example.json (optional, only logged when input example is provided and valid during model logging)

├─ serving_input_example.json (optional, only logged when input example is provided and valid during model logging)

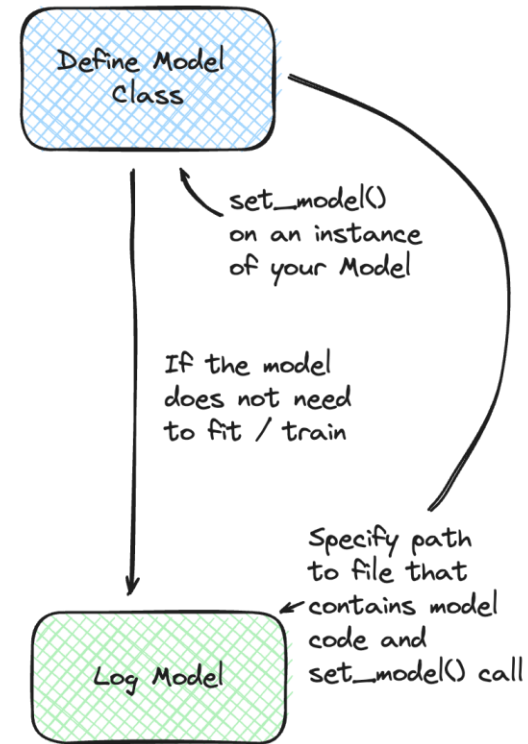
└─ environment_variables.txt (optional, only logged when environment variables are used during model inference)

Model from code

Object-based Model Logging



Logging Models from Code



Datasets tracking

Problème

Comment je sais sur quelles données ont servi à l'entraînement de mon modèle ?

Un modèle se dégrade en prod. J'aimerais aller comparer les données du temps de l'entraînement avec les nouvelles données d'aujourd'hui.

```
# For Delta tables (requires delta-lake package)  
delta_dataset = mlflow.data.from_spark(  
    spark_df, source="delta://path/to/delta/table", name="delta-table-data")
```

Datasets tracking

```
import mlflow.data
import pandas as pd

# Load your data
dataset_source_url = (
    "https://raw.githubusercontent.com/mlflow/mlflow/master/tests/datasets/wine-
    quality-white.csv"
)
raw_data = pd.read_csv(dataset_source_url, delimiter=";")

# Create a Dataset object
dataset = mlflow.data.from_pandas(
    raw_data, source=dataset_source_url, name="wine-quality-white",
    targets="quality"
)

# Log the dataset to an MLflow run
with mlflow.start_run():
    mlflow.log_input(dataset, context="training")
```

Limites de MLflow

1. MLflow sert au tracking et moins à la reproductibilité.
2. Aucune gestion des accès/authentification
3. Pas de schéma imposé : les équipes doivent trouver les conventions de nommage et de structure des expériences, models, datasets, paramètres et tags.
4. Gestion des données superficielle (métadonnées, hash et pas de stockage)
5. Pas d'orchestration native
6. Pas de détection du changement de code
7. Tracing de la consommation hardware possible depuis la version 3.2 (2025)

💡 Voir alternatives payantes [Weights & Biases \(W&B\)](#), [Neptune](#), [Comet](#)

💡 Combiner avec kedro

```
pip install kedro-mlflow
```

Neptune Service Shutdown - March 5, 2026

Following its acquisition by OpenAI, Neptune services will be permanently discontinued.

All remaining data will be deleted at shutdown and cannot be recovered.

See timelines, export instructions, and migration guides in the Transition Hub.



Librairie utile #5



LangSmith

✓ GenAI/LLM